

Práctica 3

Martínez Mejía Eduardo
Victoria Morales Ricardo Maximiliano
Torres Nava Hazel

September 25, 2026

Repositorio

El repositorio con código fuente:

<https://github.com/Concurrente-2027-1/concurrency-masters/tree/main>

La carpeta específica de la práctica 3 con el código fuente:

<https://github.com/Concurrente-2027-1/concurrency-masters/tree/main/practica3>

Ejercicio 1

(a) Para probar que tu candado funciona, crea una implementación en donde utilices un `ExecutorService` para ejecutar 400 tareas, cada tarea debe aumentar en uno un objeto `Contador`. Utiliza tu candado para 4 hilos para tener consistencia en tu `Contador`.

Para realizar esta prueba, implementé la clase `LockDoublePeterson`, utilizando un `ExecutorService` con 4 hilos para ejecutar un total de 400 tareas. Cada tarea adquiere el candado, incrementa en uno el contador compartido y registra una victoria individual para el hilo que ejecutó dicha tarea.

El objetivo de esta implementación era utilizar el candado para garantizar la exclusión mutua y, por lo tanto, mantener la consistencia del contador compartido. Sin embargo, al observar las ejecuciones obtenidas en la terminal, pude notar que existe un problema con la exclusión mutua.

En la tercera ejecución mostrada en la terminal, el valor final del contador fue de 399, mientras que la suma de las victorias registradas por los cuatro hilos fue correctamente de 400, es decir, 100 victorias por cada hilo. Esto indica que una de las operaciones de incremento se perdió.

A partir de este resultado, yo considero que en esa ejecución ocurrió una condición de carrera (*race condition*). Esto pudo suceder porque dos hilos accedieron al contador prácticamente al mismo tiempo, de manera que ambos pudieron leer el mismo valor antes de realizar el incremento y posteriormente sobrescribir el resultado del otro hilo.

Por lo tanto, aunque la implementación utiliza el candado de Peterson para controlar el acceso a la sección crítica, el resultado obtenido demuestra que el candado no logró garantizar la exclusión mutua correctamente en ese instante particular. Si el candado estuviera funcionando de manera completamente correcta, después de ejecutar las 400 tareas el valor del contador debería ser siempre 400.

```
hazel@hazel:~/Downloads/practica3/practica3$ java LockDoublePeterson
400
hazel@hazel:~/Downloads/practica3/practica3$ java LockDoublePeterson
400
hazel@hazel:~/Downloads/practica3/practica3$ java LockDoublePeterson
400
hazel@hazel:~/Downloads/practica3/practica3$ java LockDoublePeterson
400
hazel@hazel:~/Downloads/practica3/practica3$ java LockDoublePeterson
399
hazel@hazel:~/Downloads/practica3/practica3$ java LockDoublePeterson
400
hazel@hazel:~/Downloads/practica3/practica3$ java LockDoublePeterson
400
hazel@hazel:~/Downloads/practica3/practica3$ java LockDoublePeterson
400
hazel@hazel:~/Downloads/practica3/practica3$ java LockDoublePeterson
400
hazel@hazel:~/Downloads/practica3/practica3$ java LockDoublePeterson
400
hazel@hazel:~/Downloads/practica3/practica3$ java LockDoublePeterson
400
hazel@hazel:~/Downloads/practica3/practica3$ java LockDoublePeterson
400
```

(b) De alguna forma obtén el número de veces que los hilos aumentan el contador. ¿Cada uno realiza exactamente 100 tareas o hay algunos que realizan más?

Para obtener esta información, registré individualmente las victorias de cada uno de los cuatro hilos. A partir de las ejecuciones obtenidas, pude observar que cada hilo realiza exactamente 100 tareas, por lo que ningún hilo ejecuta más tareas que los demás.

Esto se debe directamente a la forma en que está programado el ciclo `for` en el método `main`. Para asignar las tareas a los cuatro hilos se utiliza la operación módulo:

```
id = i % 4
```

De esta manera, los identificadores de los hilos se van asignando de forma cíclica entre 0, 1, 2 y 3. Como existen 400 iteraciones, cada identificador aparece exactamente 100 veces.

Por lo tanto, aunque el acceso al contador presenta el problema observado en el inciso anterior, la distribución de las tareas sí es uniforme. Cada hilo recibe exactamente 100 tareas y, por lo tanto,

se registran 100 victorias para cada uno, sumando un total de 400.

```
400
Hilo 0 ha ganado 100 veces.
Hilo 1 ha ganado 100 veces.
Hilo 2 ha ganado 100 veces.
Hilo 3 ha ganado 100 veces.
hazel@hazel:~/Downloads/practica3/practica3$ java LockDoublePeterson
400
Hilo 0 ha ganado 100 veces.
Hilo 1 ha ganado 100 veces.
Hilo 2 ha ganado 100 veces.
Hilo 3 ha ganado 100 veces.
hazel@hazel:~/Downloads/practica3/practica3$ java LockDoublePeterson
399
Hilo 0 ha ganado 100 veces.
Hilo 1 ha ganado 100 veces.
Hilo 2 ha ganado 100 veces.
Hilo 3 ha ganado 100 veces.
hazel@hazel:~/Downloads/practica3/practica3$ java LockDoublePeterson
400
Hilo 0 ha ganado 100 veces.
Hilo 1 ha ganado 100 veces.
Hilo 2 ha ganado 100 veces.
Hilo 3 ha ganado 100 veces.
hazel@hazel:~/Downloads/practica3/practica3$ java LockDoublePeterson
```

(c) ¿Consideras que la implementación cumple con Justicia? Justifica tu respuesta.

Yo considero que la implementación sí cumple con la propiedad de justicia, al menos de acuerdo con el funcionamiento esperado del algoritmo de Peterson y con los resultados empíricos que obtuve.

En el algoritmo de Peterson, la variable `victim` permite resolver la competencia entre dos hilos. Cuando dos hilos intentan entrar simultáneamente a la sección crítica, el último hilo que se registra como víctima debe ceder el paso al otro hilo cuando este también está intentando entrar. De esta manera, se busca evitar que uno de los hilos quede esperando indefinidamente.

En mi implementación se utilizan tres instancias del algoritmo de Peterson para construir el candado `DoublePeterson`. Como cada instancia individual de Peterson busca garantizar la justicia entre los hilos que compiten, la combinación de estas instancias permite mantener esta propiedad para los cuatro hilos.

Además, pude comprobarlo de manera empírica al observar los resultados de las ejecuciones. Cada uno de los cuatro hilos realizó exactamente 100 tareas, por lo que ninguno obtuvo una cantidad mayor de oportunidades para ejecutar que los demás.

Por lo tanto, considero que la implementación presenta un comportamiento justo en cuanto a la distribución de las tareas. Sin embargo, es importante distinguir esta propiedad de la exclusión mutua: aunque cada hilo recibió exactamente 100 tareas, la ejecución donde el contador terminó en 399 muestra que todavía existe un problema en la protección de la sección crítica.

Ejercicio 2

(a) Con ayuda de un `ExecutorService` ejecuta 400 tareas, cada tarea debe aumentar en uno un objeto Contador. Utiliza tu candado para 4 hilos para tener consistencia en tu Contador.

Para implementar el algoritmo visto en clase se tuvieron que escribir muchas más cosas y métodos extra para crear todo el entorno de trabajo.

Se inicializaron las banderas y etiquetas (o tickets) en arreglos de tamaño 4 (pues usamos 4 hilos) definidos como **volatile** para asegurar que todas las actualizaciones que los hilos realicen a estas se reflejen en el orden correcto desde la compilación.

Se tuvo que usar la clase **ThreadLocal** para facilitar el poder darles IDs a los hilos desde 0 y en sucesión, pues la clase Thread normal les daba números distintos más difíciles de usar.

En la implementación del algoritmo referenciado, casi todo se escribió directamente, y se tuvo que agregar un método extra para comparar por orden lexicográfico las duplas de (label, ID) cada que un hilo espera a todos los turnos anteriores comparando con sus duplas.

La tarea se creó como un **Runnable**, donde cada hilo suma 1 al contador, separando la sección crítica con los candados de *Bakery* dentro de un *try* y *finally* para asegurar que siempre se libere la sección crítica, y se creó una Pool de 4 hilos mediante un **ExecutorService** para repartir la tarea, la cual repetimos las 400 veces indicadas.

Finalmente apagamos el ejecutor ya que los hilos terminen de ejecutar sus tareas, e imprimimos los resultados de la ejecución.

(b) De alguna forma obtén el número de veces que los hilos aumentan el contador. ¿Cada uno realiza exactamente 100 tareas o hay algunos que realizan más?

La ejecución del código implementado nos devuelve la siguiente impresión en la terminal:

```
Resultado final del contador: 400
El hilo 0 incrementó el contador 36 veces.
El hilo 1 incrementó el contador 18 veces.
El hilo 2 incrementó el contador 52 veces.
El hilo 3 incrementó el contador 294 veces.
```

Veamos otra ejecución:

```
Resultado final del contador: 400
El hilo 0 incrementó el contador 58 veces.
El hilo 1 incrementó el contador 84 veces.
El hilo 2 incrementó el contador 211 veces.
El hilo 3 incrementó el contador 47 veces.
```

Podemos ver que cada hilo ejecuta la tarea una cantidad de veces distinta. Por suposición, es posible que se deba a cómo se van desocupando los hilos después de ejecutar sus tareas y que uno al desocuparse más rápido termine tomando más asignaciones de la tarea.

(c) ¿Consideras que la implementación cumple con Justicia? Justifica tu respuesta.

Considero que sí, pues el hilo que llegó primero obtiene un ticket menor a los hilos que llegaron después, y el algoritmo se asegura de que sólo llegue a la sección crítica si es su turno, es decir, si tiene el ticket de menor valor. Sin embargo, si llegara a suceder que tengan el mismo número de ticket, la comparación lexicográfica que usa el algoritmo donde también compara el ID de los hilos, donde siempre pasa a la sección crítica quien tenga menor ID, se seguiría cumpliendo la precedencia, por lo que siempre entrará un hilo en el orden en el que llegó (incluso cumple con Starvation Free por lo mismo).

Ejercicio 3

Inciso a

Atomic Reference sirve para poder trabajar con objetos que requieren de cierta atomicidad, es decir, que hacen ciertas operaciones, sus métodos realizan ciertas operaciones que necesitan ser atómicas para que funcionen bien. `compareAndSet()` sirve para comparar el valor del objeto de un Atomic Reference y si se cumple cierta comparación, entonces actualiza su valor. `get()` sirve para obtener el valor de una objeto de Atomic Reference.

Inciso b

Básicamente es la implementación de bakery, se implementa el candado bakery utilizando nodos que actúan como hilos, se utiliza la biblioteca de Atomic Reference para poder manejar atomicidad y hacer que el contador cuente bien un valor de 400, es decir, aumente de uno en uno hasta llegar a 400 usando un pool de hilos de 5 hilos.

Inciso c

No va a seguir funcionando debido que al trabajar con más de un hilo, es decir, tener un sistema concurrente, necesitamos que se cumpla la propiedad de exclusión mutua o de atomicidad. Si quitamos lo relacionado a Atomic Reference, entonces perdemos la atomicidad y el algoritmo no cumple con su especificación. Lo mismo pasaba con la cola concurrente de la práctica 2, si no implementábamos candados o utilizábamos `synchronized`, entonces el algoritmo no funcionaba correctamente.

Inciso c

Consideramos que sí se implementa el algoritmo de bakery porque sí se maneja lo que son los números de turnos y las banderas de si un hilo o nodo quiere el turno o no. Sin embargo, se implementa de manera bastante diferente a lo que uno esperaría con la teoría vista en clase, pues en este código además de utilizar Atomic Reference, utiliza nodos como hilos.